Soprillo: Small, but Mechanized SAX

ARMAN AHMED, McGill University, Canada
MAX GROSS, McGill University, Canada
FRÉDÉRIC LAHAIE-BERTRAND, Université de Montréal, Canada

The semi-axiomatic sequent calculus (SAX), introduced by DeYoung, Pfenning, and Pruiksma, arises from the standard sequent calculus G_3 by replacing all noninvertible rules with axioms while leaving invertible rules unchanged. The resulting proof system is equivalent to G_3 except that standard cut-elimination fails, and is replaced by a modified form in which certain analytic cuts, called snips, are permitted to remain in normal-form proofs while preserving the subformula property. SAX admits a natural computational interpretation as a form of shared memory concurrency.

This paper presents a mechanization of the proof theory of SAX in the proof assistant Beluga. A central challenge is the need to track eligibility, a property of formula occurrences that identifies subformulas of principal formulas introduced in axioms that are eligible for a snip. We describe how eligibility is represented in Beluga as a predicate over derivations, and present mechanized proofs of the admissibility of cut and the subformula property for a representative fragment of SAX. To our knowledge, this is the first mechanization of SAX proof theory.

ACM Reference Format:
Arman Ahmed, Max Gross, and Frédéric Lahaie-Bertrand. 2026. Soprillo: Small, but Mechanized SAX. 1, 1 (April 2026), 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

The Curry-Howard isomorphism shows the deep relationship between intuitionistic logic and computation. Specifically, it establishes that proofs can be interpreted as programs and proof reduction as computation. Because of this, we can see that the different formalizations of intuitionistic logic will result in different computational interpretations [5]. Howard [8], for example, established the connection between natural deduction and the simply-typed λ-calculus. Curry [3] took a different approach, associating Hilbert-style proofs to combinators. Herbelin [7] established the system LJT based on the sequent calculus with a stoup, leading to a computational interpretation featuring explicit substitutions.

Adding to this lineage, DeYoung et al. [5] presents the semi-axiomatic sequent calculus (SAX), a formalization of intuitionistic logic which arises from by replacing all non-invertible rules in the sequent calculus with axioms.

We note that the rules that are maintaned from the standard sequent calculus (invertible rules) are the right rules from negative connectives ($A \supset B$, $A\&B$), and the left rules of positive connectives ($A \otimes B$, $\mathbf{1}$, $A \vee B$) [5]. These rules can be seen as asynchronous in that they can be applied eagerly in bottom-up proof construction. Conversely, the left rules of negative connectives and the right rules of positive connectives may be postponed [5]. These properties, along with SAX's cut elimination, form the first seeds of intuition for its computational interpretation [5].

Authors' Contact Information: Arman Ahmed, arman.ahmed@mail.mcgill.ca, McGill University, Montréal, Canada; Max Gross, max.gross@mail.mcgill.ca, McGill University, Montréal, Canada; Frédéric Lahaie-Bertrand, frederic.lahaie-bertrand@umontreal.ca, Université de Montréal, Montréal, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/4-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

$$\frac{}{\Gamma, A \vdash A}\ \mathsf{Id}_A \qquad \frac{\Gamma \vdash A \quad \Gamma, A \vdash C}{\Gamma \vdash C}\ \mathsf{Cut}_A$$

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \supset B}\ \supset R \qquad \frac{}{\Gamma, A, A \supset B \vdash B}\ \supset L^0$$

$$\frac{}{\Gamma, A, B \vdash A \otimes B}\ \otimes R^0 \qquad \frac{\Gamma, A \otimes B, A, B \vdash C}{\Gamma, A \otimes B \vdash C}\ \otimes L$$

$$\frac{}{\Gamma \vdash \mathbf{1}}\ \mathbf{1}R^0 \qquad \frac{\Gamma, \mathbf{1} \vdash C}{\Gamma, \mathbf{1} \vdash C}\ \mathbf{1}L$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \,\&\, B}\ \&R \qquad \frac{}{\Gamma, A \,\&\, B \vdash A}\ \&L_1^0 \qquad \frac{}{\Gamma, A \,\&\, B \vdash B}\ \&L_1^0$$

$$\frac{}{\Gamma, A \vdash A \vee B}\ \vee R_1^0 \qquad \frac{}{\Gamma, B \vdash A \vee B}\ \vee R_2^0 \qquad \frac{\Gamma, A \vee B, A \vdash C \quad \Gamma, A \vee B, B \vdash C}{\Gamma, A \vee B \vdash C}\ \vee L$$

Fig. 1. The Semi-Axiomatic Sequent Calculus (Sax) [5].

While the resulting system is sound and complete with respect to the sequent calculus, its proof theory contains many more intricacies than that of the sequent calculus. For example, the system, as articulated in figure 1, is not cut-admissible [5]. Though, we can restrict the system to a form of analytic cuts, snips, and retain a (general) cut-free proof [5]. These snips require that the cut formula in at least one of the premises originate from an axiom [5]. With their addition, SAX also satisfies the subformula property [5]. In order to track the origins of each formula, [5] introduces a labelling system, marking formulas $y^* : A$ as eligible for a snip. We can then propogate this through the proof by marking a variable as eligible in the conclusion if it is eligible in at least one of its premises [5]. Since the application of axioms can contain antecedents that are irrelevant (antecedents that are not strictly needed for the application of said axiom) we must also track these irrelevant formulas by marking them $x^0 : B$. We mark a variable as irrelevant in the conclusion if it is irrelevant in both of its premises [5].

$$\frac{}{\Gamma^0,\ x : A \vdash y : A}\ \mathrm{Id}_A \qquad \frac{\Gamma \vdash x : A \quad \Gamma,\ x : A \vdash z : C}{\Gamma \vdash z : C}\ \mathrm{Cut}_A$$

$$\frac{\Gamma \vdash x^* : A \quad \Gamma,\ x : A \vdash z : C}{\Gamma \vdash z : C}\ \mathrm{Snip}_A^1 \qquad \frac{\Gamma \vdash x : A \quad \Gamma,\ x^* : A \vdash z : C}{\Gamma \vdash z : C}\ \mathrm{Snip}_A^2$$

$$\frac{\Gamma,\ y : A \vdash z : B}{\Gamma \vdash x : A \supset B}\ \supset R \qquad \frac{}{\Gamma^0,\ y^* : A,\ x : A \supset B \vdash z^* : B}\ \supset L^0$$

$$\frac{}{\Gamma^0,\ y^* : A,\ z^* : B \vdash x : A \otimes B}\ \otimes R^0 \qquad \frac{\Gamma,\ x : A \otimes B,\ y : A,\ z : B \vdash w : C}{\Gamma,\ x : A \otimes B \vdash w : C}\ \otimes L$$

$$\frac{}{\Gamma^0 \vdash x : 1}\ 1R^0 \qquad \frac{\Gamma,\ x : 1 \vdash w : C}{\Gamma,\ x : 1 \vdash w : C}\ 1L$$

$$\frac{\Gamma \vdash y : A \quad \Gamma \vdash z : B}{\Gamma \vdash x : A \,\&\, B}\ \&R \qquad \frac{}{\Gamma^0,\ x : A \,\&\, B \vdash y^* : A}\ \&L_1^0 \qquad \frac{}{\Gamma^0,\ x : A \,\&\, B \vdash z^* : B}\ \&L_2^0$$

$$\frac{}{\Gamma^0,\ x^* : A \vdash z : A \vee B}\ \vee R_1^0 \qquad \frac{}{\Gamma^0,\ y^* : B \vdash z : A \vee B}\ \vee R_2^0$$

$$\frac{\Gamma,\ z : A \vee B,\ x : A \vdash w : C \quad \Gamma,\ z : A \vee B,\ y : B \vdash w : C}{\Gamma,\ z : A \vee B \vdash w : C}\ \vee L$$

Fig. 2. Labelled SAX [5].

This added bookkeeping is precisely what makes the proof theory of SAX an interesting target for mechanization. To prove cut-admissibility, while we can inherit the general approach of a nested structural induction on both derivations and the cut formula from Pfenning [11], the sensitivity of snips to the eligibility of the cut-formulas requires us to reason about the eligibility of each cut formula as well as their derivation. This necessitates that a mechanization of SAX tracks eligibility, and that we can reason about it in our case analysis.

As such, we report on a mechanization of the proof theory of Soprillo, a fragment of SAX in Beluga, a proof assistant with its foundations in contextual modal logic and the Logical Framework. Beluga's explicit handling of contexts is particularly suited to mechanizing SAX, precisely because we need to reason about variables in contexts during our proof. Further, since it is an extension of LF, we can build on the existing literature regarding mechanizing similar systems and proofs in the literature [2] [11]. This, to the best of our knowledge, is the first mechanization of SAX and its proof theory. Specifically, we present:

- A mechanization of SAX
- A mechanized proof of the translation theorem between SAX and the sequent calculus
- Mechanized proofs of cut-admissibility, cut-elimination, and the subformula property

2 Motivation

3 Contributions

4 Related Work

4.1 SAX and Related Systems

Francalaza et. al. [6], present an alternate formulation of SAX embedded in adjoint logic. The resulting computational interpretation models message-passing concurrency: each variable in their system becomes a channel, whose properties are modeled by its adjoint modality, and each consequent becomes a provider of said channels [6]. They present an implementation of this language in Go [6]. Logically then, Pfenning and Pruiksma [12] present SAX with recursion as a unifying framework between message-passing and shared memory. Specifically, they show that the shared-memory semantics and the message-passing semantics have a strong bisimulation on well-typed terms [12].

DeYoung and Pfenning [4] present SNAX, a modification of the standard type theory of SAX that accounts for data layout in shared memory concurrency. It achieves this, in part, by making eligibility a first-class part of the system, rather than a label, as described above; they also add recursion to the system [4]. Ng and Pfenning [10] present SNAX with reuse, a single-threaded language based on SNAX that models memory reuse.

Boyland and Pfenning [1] present an additional adjoint-logical formulation of SAX with bidirectional typing rules. They further show a compilation from adjoint natural deduction [9] to their adjoint system, building the groundwork for a compiler of adjoint-functional programs [1]. Somayyajula and Pfenning [14] applies a type refinement system to SAX, allowing them to reason about the partial correctness of SAX programs. Their framework also allows for the synthesis of SAX with Hoare logic [14].

4.2 Related Mechanizations

Pfenning [11] presents an elegant proof for cut-elimination in the sequent calculus with the same structure as in [5], and our mechanization. He also presents a formalization of this proof in LF [11]. Crary [2] describes a method for encoding linearity in LF without any linear or modal extensions. We used his techniques to in our representation of SAX in Beluga to properly represent eligible formulas in our system. These act similarly to propositions in linear logic as every eligible formula must be used in a SAX proof. We also drew on the techniques described in [13] to properly represent SAX in Beluga.

5 Conclusion

We report on a mechanization of SAX and a mechanization of the proof of its translation to the sequent calculus. We also mechanize the proofs of cut-elimination, cut-admissibility, and the subformula property for Soprillo, a fragment of SAX containing only snips, implication, and the identity rule.

5.1 Future Work

The most obvious avenue for future work is to extend our mechanizations to fragments larger than Soprillo. While extending these proofs to the complete SAX system may be infeasible, mechanizations that include $\otimes$ would be of great use in showing the adaptability of our representations and proof techniques used. In a similar vein, a mechanization of SAX within adjoint logic would allow for similar meta-theoretic guarantees regarding the message-passing interpretation of SAX. Further, a mechanization of SNAX should follow relatively easily from our mechanization.

Another natural step is a mechanization of the shared memory interpretation of SAX, along with its properties including progress and termination. [5] presents a logical relations argument for the latter, which are particularly amenable to mechanization. We suspect that the dynamic semantics of the shared memory interpretation, which takes the form of multiset rewriting rules on configurations (allocated memory cells) can be represented as ordered lists in Beluga. While we intended to explore this path further, our initial challenges at properly representing eligibility in Beluga stalled the project significantly.

References

[1] Joanna Boyland and Frank Pfenning. 2025. Compiling adjoint natural deduction to the semi-axiomatic sequent calculus. Abstract, 9th International Workshop on Trends in Linear Logic and its Applications (TLLA 2025), affiliated with FSCD 2025, Birmingham, UK, 19–20 July 2025. https://lipn.univ-paris13.fr/TLLA/2025/abstracts/06-boyland-pfenning.pdf

[2] Karl Crary. 2010. Higher-order representation of substructural logics. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming* (Baltimore, Maryland, USA) *(ICFP '10)*. Association for Computing Machinery, New York, NY, USA, 131–142. doi:10.1145/1863543.1863565

[3] H. B. Curry. 1934. Functionality in Combinatory Logic*. *Proceedings of the National Academy of Sciences* 20, 11 (1934), 584–590. arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.20.11.584 doi:10.1073/pnas.20.11.584

[4] Henry DeYoung and Frank Pfenning. 2023. Data Layout from a Type-Theoretic Perspective. *Electronic Notes in Theoretical Informatics and Computer Science* 1 (feb 2023). doi:10.46298/entics.10507 Proceedings of MFPS XXXVIII.

[5] Henry DeYoung, Frank Pfenning, and Klaas Pruiksma. 2020. Semi-Axiomatic Sequent Calculus. In *5th International Conference on Formal Structures for Computation and Deduction (FSCD 2020) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 167)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 29:1–29:22. doi:10.4230/LIPIcs.FSCD.2020.29

[6] Adrian Francalanza, Gerard Tabone, and Frank Pfenning. 2024. Implementing a Message-Passing Interpretation of the Semi-Axiomatic Sequent Calculus (Sax). In *Coordination Models and Languages - 26th IFIP WG 6.1 International Conference, COORDINATION 2024, Held as Part of the 19th International Federated Conference on Distributed Computing Techniques, DisCoTec 2024, Groningen, The Netherlands, June 17-21, 2024, Proceedings (Lecture Notes in Computer Science, Vol. 14676)*. Springer, 295–313. doi:10.1007/978-3-031-62697-5_16

[7] Hugo Herbelin. 1995. A Lambda-Calculus Structure Isomorphic to Gentzen-Style Sequent Calculus Structure. In *Computer Science Logic (CSL '94) (Lecture Notes in Computer Science, Vol. 933)*, L. Pacholski and J. Tiuryn (Eds.). Springer, Kazimierz, Poland, 61–75.

[8] W. A. Howard. 1980. The Formulae-as-Types Notion of Construction. In *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Academic Press, 479–490. Original unpublished note from 1969.

[9] Junyoung Jang, Sophia Roshal, Frank Pfenning, and Brigitte Pientka. 2024. Adjoint Natural Deduction (Extended Version). arXiv:2402.01428 [cs.LO] https://arxiv.org/abs/2402.01428 Submitted on 2 Feb 2024.

[10] Daniel Ng. 2024. Memory Reuse in Linear Functional Computation. https://www.cs.cmu.edu/~fp/courses/15417-s25/misc/Ng24.pdf Accessed: 2026-04-21.

[11] Frank Pfenning. 2000. Structural Cut Elimination: I. Intuitionistic and Classical Logic. *Information and Computation* 157, 1–2 (2000), 84–141. doi:10.1006/inco.1999.2832

[12] Frank Pfenning and Klaas Pruiksma. 2023. Relating Message Passing and Shared Memory, Proof-Theoretically. In *Coordination Models and Languages*, Sung-Shik Jongmans and Antónia Lopes (Eds.). Springer Nature Switzerland, Cham, 3–27.

[13] Chuta Sano, Ryan Kavanagh, and Brigitte Pientka. 2023. Mechanizing Session-Types using a Structural View: Enforcing Linearity without Linearity. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 235 (Oct. 2023), 26 pages. doi:10.1145/3622810

[14] Siva Somayyajula and Frank Pfenning. 2023. Dependent Type Refinements for Futures. *Electronic Notes in Theoretical Informatics and Computer Science* Volume 3-Proceedings of... (Nov. 2023). doi:10.46298/entics.12286

[15] Tesla Zhang, Sonya Simkin, Rui Li, Yue Yao, and Stephanie Balzer. 2025. A Language-Agnostic Logical Relation for Message-Passing Protocols. arXiv:2506.10026 [cs.PL] https://arxiv.org/abs/2506.10026